Project 2 - Data Link Layer ARQ Protocols

Objective:

After this experiment, the students should understand:

i. How different retransmission (ARQ) protocols work.

ii. The effect of different parameters on the performance of the protocols, including channel capacity, propagation delay, size of data frame, size of ack frame, and error ratio.

Programming languages: this experiment can be done using C, C++, or Python.

1. Overview:

In the OSI model, the *data link layer (layer 2)* handles data transfer over a single, bi-directional communication link without relays or intermediate nodes. The link can be point-to-point, broadcast, or switched. The data link layer is made of 2 components. The lower one is the MAC (Multiple-Access Control) sub-layer that manages multipoint links (this means that if the link is point-to-point, there is no MAC protocol). The upper one is called LLC (Logical Link Control). This project focuses on LLC and hence assumes a point-to-point link. On the sending side the data link layer encapsulates protocol data units (PDUs), also known as packets, coming from the network layer (layer 3) into a layer 2 PDU called a frame, adds its header including error control bits (e.g., CRC), and sends the whole frame to the physical layer (layer 1). The data link layer supervises the physical transmission of frames by the physical layer, performs error control over the link (at least error detection), and possibly initiates retransmission when errors are detected and cannot be corrected. Therefore, in its most complete and complex form, the data link layer provides *a reliable point-to-point* communication service between two adjacent nodes, with each corresponding layer 2 peer process capable of sending and receiving frames. See Figure 1.

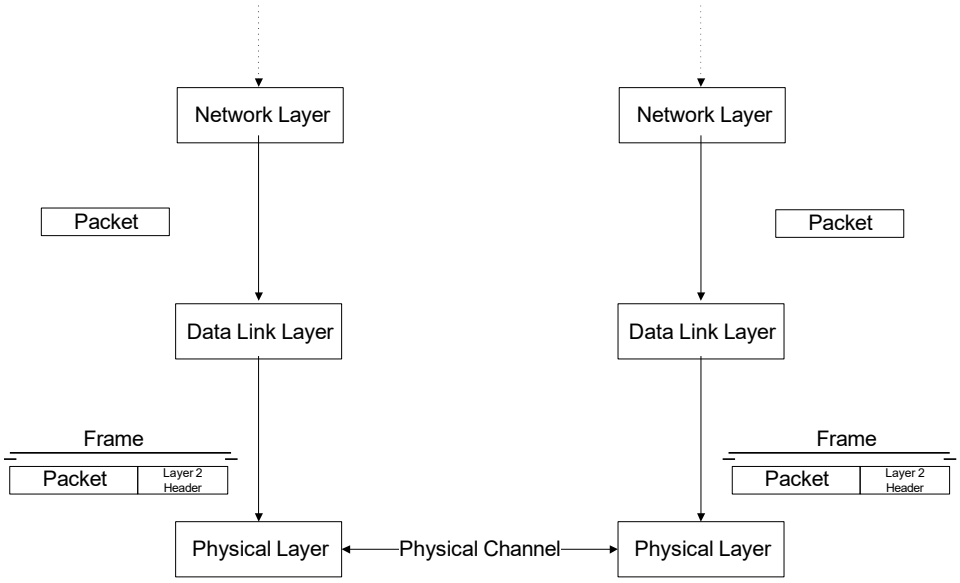


Figure 1 Relationship between packets and frames

There are two main types of error control: *Forward Error Correction (FEC) and Automatic Repeat reQuest (ARQ)*. FEC will embed *error correction code* into the frame and the receiver will try to correct the errors, while ARQ will embed *error detection code* in the frame and the receiver will detect errors and request retransmission if errors occur. Although ARQ involves extra overhead for retransmission requests and retransmitting frames that were received with errors, the total overhead for data links with reasonable error rates is often far less than it would be with FEC.

It is interesting to note that ARQ is not only applicable to the data link layer, but also to the transport layer and hence what you are learning here is crucial for your understanding of TCP. The main difference between an ARQ protocol at layer 2 and layer 4 is that at layer 2, the sender and the receiver are connected through a physical channel which can corrupt, delay or lose frames while at layer 4, the sender and the receiver are connected through a network which can not only corrupt or lose layer 4 PDUs (called segments) but also introduce more delay and delay variation than a channel and deliver segments out-of-order. In the following, we only consider the case of layer 2 ARQ protocols.

2. Background Material:

In this experiment, we will investigate one widely used ARQ protocol, namely, *Alternating Bit Protocol (ABP)*. We will focus on one direction for data transmission. In general data flows in both directions. Referring to Figure 2, A is the side sending data and B is the side receiving it and sending acknowledgements (ACK).

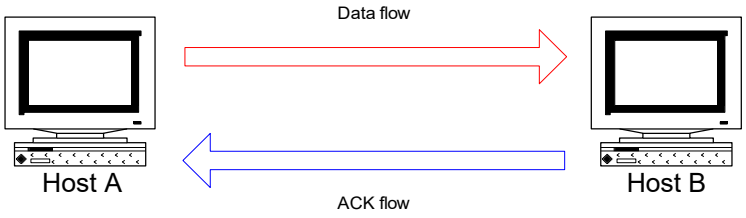


Figure *2*

2.1 ABP

In ABP, the sender (sender here refers to the data link layer process) sends packets in data frames numbered 0 and 1 (this number is called a *sequence number*) alternatively and starts a timeout after each frame it sends. (The frame is called a data frame because it actually contains data from layer 3). The sender has a buffer able to contain 1 packet in which the packet currently being sent is kept until the sender is sure that it has been received correctly by the other side. The sender sends one packet at a time, i.e., it does not send packet #j before being completely sure that packet #(j-1) has been correctly received. When the sender has completely finished with packet #(j-1), i.e., it has received a non-corrupted ACK for it, it empties its buffer and requests a new packet from the upper layer. The upper layer may not have a packet to send right away but is then aware of the availability of the layer 2 sending process.

The channel can corrupt or lose a data frame or an ack frame. The receiver keeps a counter NEXT_EXPECTED_FRAME which represents the frame sequence number that it is expecting. A correct frame for the receiver is a frame that is received without error. If the receiver receives a correct frame whose sequence number is equal to NEXT_EXPECTED_FRAME, it interprets it as a new frame with a new packet. It sends the packet to Layer 3, increases its counter NEXT_EXPECTED_FRAME by 1 (modulo 2) and acknowledges the receipt of the frame by

sending an acknowledgement (ACK) which is encapsulated in a frame from B to A. This frame can either be a data frame from B to A or a special frame that only contains the ACK. We will assume the latter in the following. This ACK frame carries an acknowledgement number RN equal to the updated value of NEXT_EXPECTED_FRAME. If the receiver receives a frame in error or a correct frame with an unexpected SN number (i.e., not equal to NEXT_EXPECTED_FRAME), it will not update the counter value NEXT_EXPECTED_FRAME and will send an ACK with the current value of NEXT_EXPECTED_FRAME as RN.

The sender maintains a counter NEXT_EXPECTED_ACK which is set to SN+1 (modulo 2) where SN is the sequence-number of the latest frame sent by the sender. It will wait for the receiver's ACK with the right ACK number (i.e., RN being equal to NEXT_EXPECTED_ACK) before emptying its buffer of the current packet, increasing SN and NEXT_EXPECTED_ACK by 1 (modulo 2) and informing the upper layer that it is ready to send a new packet. It will send a new packet in a new numbered frame as soon as the layer 3 of A has one to send. More precisely, when it receives a frame, if it is correct, it checks the value of RN. If RN = NEXT_EXPECTED_ACK, it does what was explained earlier in this paragraph, otherwise if the frame is correct but RN is not equal to NEXT_EXPECTED_ACK or if the frame contains errors, 2 options are possible. In the first option, the sender does not do anything, while in the second option, it resends the packet in the buffer in a new data frame with the same SN. If the sender does not receive an acknowledgement before the frame's timeout, either because the ACK was not sent by the receiver or was lost or because the timeout was too short, the sender will retransmit the same packet in a new frame carrying the same sequence number SN as the original frame.

3. Simulation Framework:

Figure 3 shows the system that you are going to simulate:

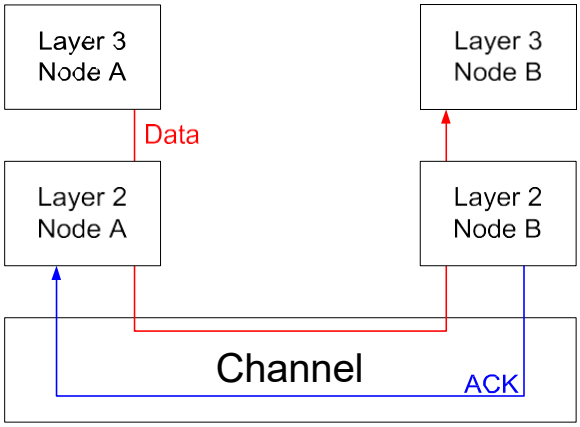


Figure 3 Model for simulation

We will assume that data always flows from node A (sender) to node B (receiver). You should bear in mind that in reality both nodes could be a sender of data and both could be a receiver of data. The channel is always bi-directional, even in this model, because data will go from A to B and ACK will go from B to A. Furthermore, we will assume that Layer 3 of node A ALWAYS has packets to send. Your simulator should be built based on all these assumptions.

We will use the DES (Discrete Event Simulation) framework introduced in Project 1 to implement our simulator. A queue called the Event Scheduler (ES) will be used to store the events in sequence and a function will be used to return the event at the head of the event scheduler (the next earliest happening event), which we call the next event. The structure of an event has to be defined, with fields (properties) which are associated with it. Among other things, an event always has a *time* field, telling us when it occurred. There can be multiple ways in which the DES simulator can be designed. As a guideline, we will mention one particular approach and describe the important functional elements of the design. We expect you to follow this guideline. Fig. 4 shows the different functional components of this guideline design. Next, we will describe this design and the behaviors of the sender, receiver and the channel.

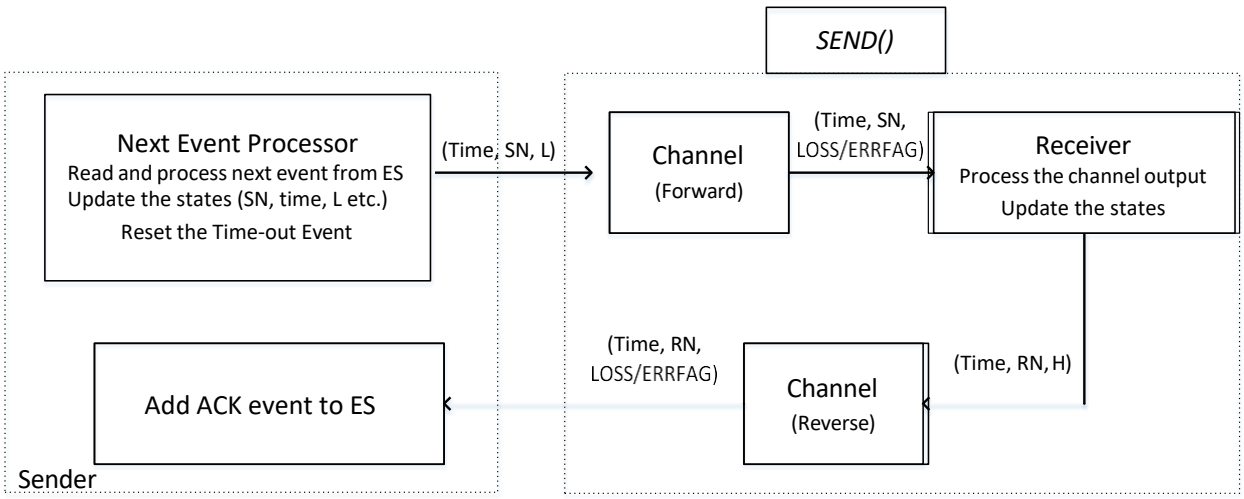


Figure 4: The functional diagram

First you need to define an event, which can be of one of two types: TIME-OUT and ACK. A TIME-OUT event has only one field: *time*. An ACK event has two extra fields: an associated RN, and its status, i.e., is it in error or not. So, you can define an event as a structure with four fields, namely *type* (TIME-OUT or ACK), *time, error-flag and sequence-number*. An event scheduler (ES) is a time-ordered sequence of such events. It is similar to the event scheduler that you designed in Project 1.

We will briefly discuss the main components of the simulator, by focusing on the ABP protocol.

Details of the ABP Simulator

1) **ABP Sender**: The sender maintains two counters, namely SN and NEXT_EXPECTED_ACK. SN is initialized to 0 and NEXT_EXPECTED_ACK is set to 1. In the beginning, the sender also chooses a value of time-out Δ which is kept constant throughout the simulation. The choice of Δ affects the protocol performance greatly. The *current time* t_c is initialized to 0. The sender generates a packet of length L = H + *l*, where H is the fixed length of the frame header and *l* is the packet length. Normally, packets have variable packet lengths. However, in this project, we assume that all packets are of the same length. The sender stores this packet in its (only) buffer.

We assume (realistically) that the sender also knows the channel capacity C (in bits/sec). L/C is the amount of time required for the frame to be completely sent into the channel, i.e., the transmission delay. The sender sends the packet in a frame with sequence number SN, which is completely transferred to the channel by time $t_c + L/C$, and at this point it registers a TIME- OUT event in the ES (i.e., a time-out at time $t_c + L/C + \Delta$). The sender then calls a function that implements *together* 1) the *forward channel* (from the sender to the receiver), 2) the *receiver* and 3) the *reverse channel* (from the receiver to the sender). Let this function be called *SEND()*. It returns as a value an event that has to be registered in the ES (See Fig. 4). Occasionally, a frame (on the forward channel) or an ACK (on the reverse channel) gets lost, in which case the function returns a NIL event which will not be registered in the ES. Otherwise, the return value of *SEND()* corresponds to an ACK event, which has a type: it can either come with or without error, and has a sequence-number RN. The sender reads the ES to get the *next event*. The reading process also updates the value of current time t_c, and removes the event from ES. If the next event is a TIME-OUT event (which means either the frame or the ACK was lost, or that the ACK arrived after the time-out), the same packet with the same SN is sent to the channel in a new frame. On the other hand, if the next event is an ACK without error and RN is equal to NEXT_EXPECTED_ACK, the sender knows its packet has been received correctly, so it increments SN and NEXT_EXPECTED_ACK by 1 (mod 2), generates a new packet, and sends a new frame. Note that there can be only one time-out in the ES, and hence as soon as the frame is completely passed on to the forward channel (i.e., at $t_c + L/C$), any outstanding time-outs in ES have to be purged and a new time-out at $t_c + L/C + \Delta$ has to be registered. Event processing takes non-zero time (for example, sending a frame takes L/C time), and we will assume that once the sender starts processing an event, it does not get interrupted by another event. The events are processed sequentially without interruptions.

2) **The channel** is characterized by its *propagation delay* τ where τ is assumed to be a constant quantity. A data frame (for the forward channel) or an ACK (for the reverse channel) can be lost with a probability P_{LOSS} (which, in general, can be different for a frame and an ACK). A lost frame/ACK never appears on the other end of the channel. With a probability $(1-P_{LOSS})$, the frame/ACK arrives at the other end of the channel, with two possibilities: a) it arrives in error (frame ERROR), b) it arrives without any error (frame NOERROR). These three events (frame

LOSS, frame ERROR and frame NOERROR) are mutually dependent and usually have intricate relationships. To simplify the relationships among these outcomes, we assume that the channel has a given bit error rate BER (the probability that a bit arrives in error, considered independently from one bit to another) and we *arbitrarily* consider a frame experiencing 5 or more bit errors to be a lost frame. If a frame experiences from 1 to 4 bits in error, it is considered to be a frame in ERROR. When all bits arrive correctly, we consider this a successful arrival of the frame (NOERROR). We can compute these probabilities for a frame of length L as follows.

$$P_{NOERROR} = (1 - BER)^L$$

$$P_{ERROR} = \sum_{k=1}^{4} \binom{L}{k} BER^k (1 - BER)^{(L-k)}$$

$$P_{LOSS} = 1 - P_{NOERROR} - P_{ERROR}$$

Note that we do not expect you to use the above stated formulas to generate the NOERROR, ERROR or LOSS event. We expect you to run L iterations. In each iteration, you generate 0 with probability BER, and 1 with probability 1-BER. You then count the number of zeroes. The number of zeroes represents the number of bits in error and thus corresponds to one of the three events. Functionally speaking, the channel will take the time t_c, SN and the length of the frame/ACK as inputs and will either return nothing (NIL) if the frame/ACK is lost or return the time (after adding the delay τ), a flag representing whether the frame/ACK encountered an error or not, as well as the SN (which is unchanged).

3) **ABP Receiver**: The receiver maintains a NEXT_EXPECTED_FRAME counter, initialized to 0. The current time t_{cs} is also initialized to 0. The receiver acts on the return values of the forward channel. It updates its current time t_{cs} based on the channel return value. If the channel returns a frame with no errors, it checks the frame's SN and if it is equal to the NEXT_EXPECTED_FRAME, it counts it as a new and successfully delivered packet, which it sends to Layer 3 (in our case, it just increments a counter). This successful delivery is accompanied by increasing NEXT_EXPECTED_FRAME by 1 (mod 2). In all cases, it sends an

ACK to the reverse channel of fixed length H, with the sequence number RN set to NEXT_EXPECTED_FRAME.

Note that the return values from the reverse channel will be registered in the ES as described earlier. The forward and the reverse channel are functionally equivalent and hence should be implemented by one function definition. Also, it is important to make sure that correct tracking of time is carried out at both the sender and receiver implementation. As said already, the forward channel, receiver and the reverse channel should be wrapped into one functional block, to be used by the sender.

4. Experiment:

A simulation experiment will comprise of the following parameters.

1) Sender-side parameters: H, l, Δ
2) Channel parameters: C, τ, BER
3) Experiment duration in terms of *number of successfully delivered packets* to be simulated These parameters will affect the performance of an ARQ protocol. In this experiment, you will need to investigate how a change in one or more of these parameters affects the throughput of an ARQ protocol. We will assume that the size of the frame header (which is also the size of an ACK) is fixed to H=54 bytes and the data packet payload length is fixed to l=1500 bytes (i.e., a data frame has a constant size of H + l).

Question 1 (*ABP*)

Implement the ABP sender, channel and receiver as described above using the first option (namely the sender does nothing when it receives an ACK in error or a correct ACK with an RN not equal to NEXT_EXPECTED_ACK). Call this simulator *ABP*. Also, bundle the forward channel, receiver and the reverse channel components into one procedure. Also, implement an event scheduler, with a function for registering an event and a function for de-queueing the earliest event. With the necessary components in place, integrate them to implement the simulator. A skeleton coding framework is provided in C on D2L. Please work to complete the framework. It is also fine if you choose to use C++ or Python, then you need to re-implement the framework in your chosen language.

Accompany the snapshots of your codes in your report with comments and brief documentation explaining the key components of your implementation. **(50 points)**

1) Consider the scenario with **BER = 0**. Use your simulator to compute the throughput (bits/sec) as a function of Δ for C = 5Mb/s, and two values of 2τ (10ms and 500ms). Note that while computing the throughput, you should not count the header bits. Simulate at least 10,000 successfully delivered packets and use the same methodology as in Project 1 to obtain your results (i.e., you simulate for 5,000 successfully delivered packets record the throughput and then simulate for 5,000 more and see if the results are comparable). Take the values of Δ from the set {2.5τ, 5τ, 7.5τ, 10τ, 12.5τ} **(30 points)**

2) Now, repeat the set of experiments in (i) with **BER = 1.0e-5** and **BER = 1.0e-4**. **(20 points)**

Summary of result data

Please use the following table to show the throughput results that you obtain in 1) and 2)

Δ/τ	**2τ = 10ms**			**2τ = 500ms**		
	BER=0.0	BER=1e-5	BER=1e-4	BER=0.0	BER=1e-5	BER=1e-4
2.5	a1	a2	a3	a4	a5	a6
5	b1	b2	b3	b4	b5	b6
7.5	c1	c2	c3	c4	c5	c6
10	d1	d2	d3	d4	d5	d6
12.5	e1	e2	e3	e4	e5	e6